Как учат разработке?

Показывают код.

А спецификации?

SDD Trainer — защита проектной работы · курс «ИИ-агенты» · Standard Data · 13.11.2026

ПРОЕКТНАЯ РАБОТА · ФАЗА 2 ЗАВЕРШЕНА

SDD Trainer

ИИ-тренер по Spec-Driven Development — GigaChat · LangGraph · MCP

249
тестов — suite green

78.35%
coverage (гейт курса — 40%)

7
MCP-инструментов
stdio + Streamable HTTP

Алекс · дедлайн сдачи — 13 ноября 2026

SDD — методология без тренера

Spec-Driven Development: спецификация — **источник истины** (план, дельты, критерии приёмки). Учить её приходится по документации — и это не работает:

Нет калибровки

Курс не знает, что ученик уже умеет, — все проходят одно и то же

Нет обратной связи

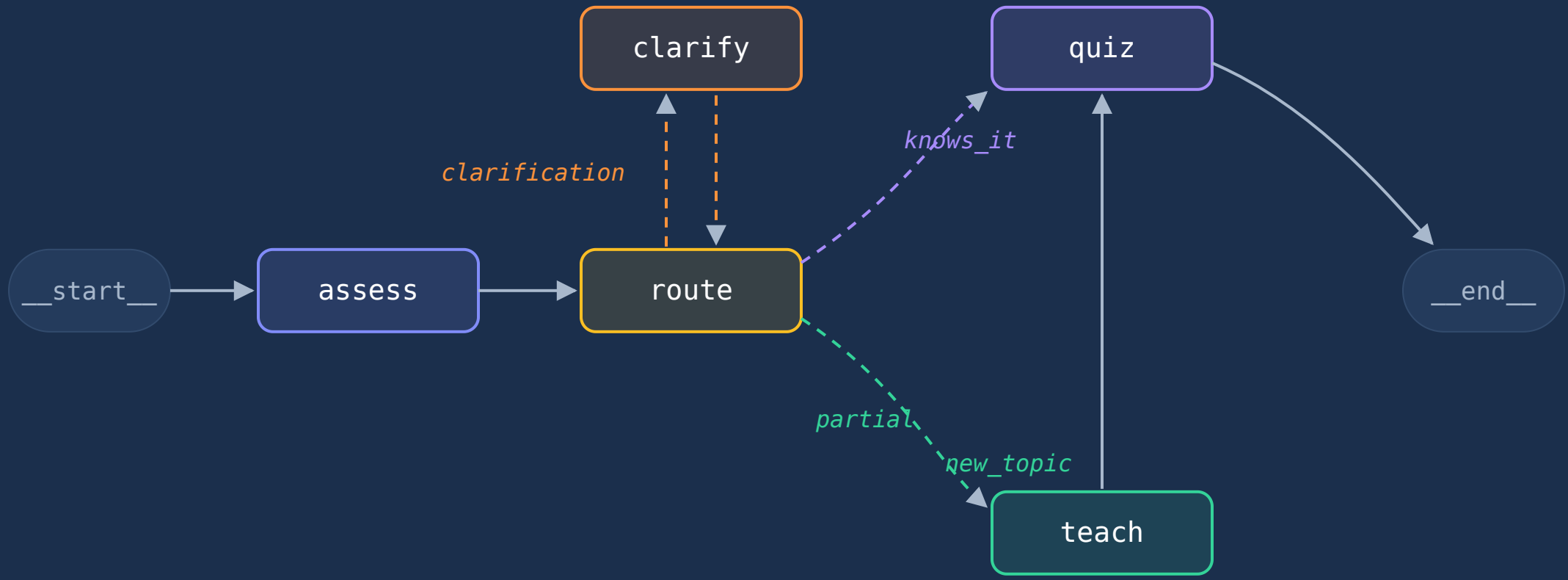
Непонятно, где именно пробелы, — доки не отвечают на вопросы

Нет проверки

«Прочитал» не значит «понял», — понимание никто не проверяет

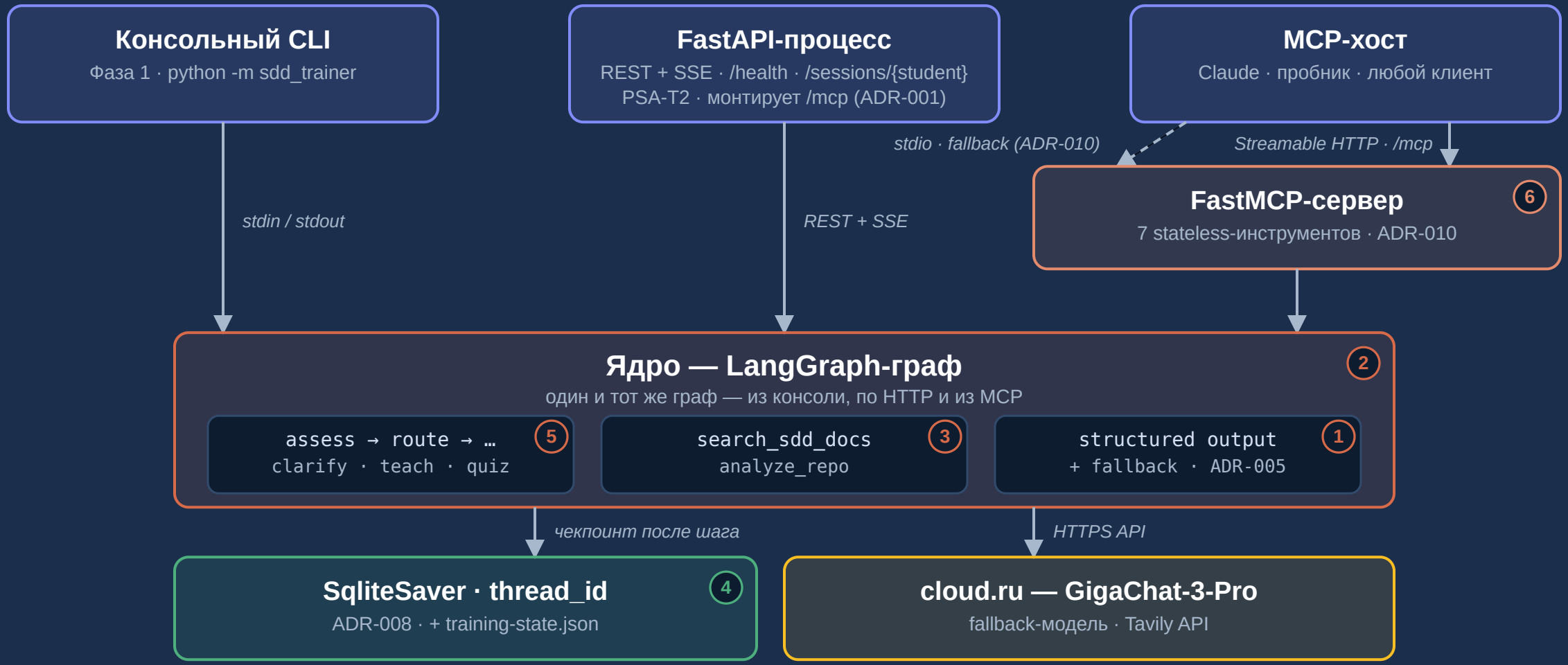
Нужен тренер, который **адаптируется под уровень** и **проверяет понимание**.

LangGraph-граф с четырьмя маршрутами



Структура — дамп скомпилированного графа (`draw_mermaid()`) · решение через structured output: `RouteDecision` (ADR-005) · knows_it → квиз сразу · partial → урок по пробелам · new_topic → урок с нуля · низкая уверенность → уточняющий цикл

Одно ядро — три входа



① GigaChat · ② LangGraph · ③ ≥2 инструмента · ④ память · ⑤ роутинг ≥3 путей · ⑥ MCP — критерии курса на компонентах

Один уровень абстракции (C4 container) · стрелки подписаны протоколами · stdio — fallback (ADR-010) · цвета: интерфейс / транспорт / ядро / хранение / внешний мир

Один сеанс — от калибровки до зачёта

🦊 ТРЕНЕР · КАЛИБРОВОЧНЫЙ ВОПРОС

Опиши своими словами, что такое OpenSpec: зачем в репозитории папки specs/ и changes/ и чем change proposal отличается от стабильной спецификации. Если тема незнакома — так и скажи, это нормально.

УЧЕНИК

Видел проект с OpenSpec: в specs/ лежат стабильные спецификации требований, в changes/ — предложения изменений. Но устройство change proposal (proposal.md, design.md, tasks.md) не разбирал, на практи…

ВЕРДИКТ: PARTIAL

уверенность **0.85**

Маршрут: partial — Ученик демонстрирует общее понимание различий между папками и статусами спецификаций, но признаёт пробелы в деталях cha…

цитаты дословные из транскрипта; длинные сокращены (…)

КВИЗ ЗАЧТЁН

F1-счёт **1.00 (порог 0.60)** · зачтён

Прогресс записан: · openspec-basics: счёт 1.00, серия 7, уроков завершено 7

Дословный транскрипт живого прогона 03.11.2026 · MCP stdio · GigaChat-3-Pro

Критерии курса 1–5 — закрыты кодом

№	КРИТЕРИЙ	ГДЕ ЖИВЁТ	ДОКАЗАТЕЛЬСТВО
1	GigaChat (cloud.ru)	`core/llm_manager.py`	OpenAI-совместимый API, structured output, fallback-модель
2	LangGraph	`core/graph.py`	StateGraph, условные рёбра, компиляция с чекпоинтером
3	≥ 2 инструмента	`core/tools.py`	search_sdd_docs (живой Tavily) + analyze_repo
4	Память	`core/memory.py`	SqliteSaver + thread_id — рестарт ничего не теряет
5	Роутинг ≥ 3 путей	`core/graph.py`	**4 маршрута**: knows_it · partial · new_topic · clarification

```
# ADR-005: ответ — только tool call, free-text не парсится
        tool = self._structure_tool(structure)
        completion = self._complete(
            messages=messages,
            tools=[tool],
            tool_choice={
                "type": "function",
                "function": {"name": tool["function"]["name"]},
            },
```

Фрагменты дословные, с номерами строк — на дашборде «Код по критериям»

MCP-слой: один вызов = один шаг графа




```
result = graph.invoke(Command(resume=answer), config)
# шаг графа до следующего interrupt —
# payload возвращается клиенту
```

Почему это не «обёртка ради обёртки»

Инструменты **stateless**: student — единственный ключ, чекпоинт — единственное состояние. Рестарт сервера ничего не теряет. stdout чист — принты только в stderr (канарейка — реальный stdio-subprocess в интеграционном тесте).

Критерий 6 закрыт буквально: FastAPI-процесс с `/mcp` (ADR-001) — REST, SSE и MCP в одном процессе; stdio — fallback.

Сервер убит посреди квиза — сессия выжила

```
$ python mcp_probe.py --student verify-m3b  # часть 1
[клиент] start_session(student="verify-m3b")
…калибровка → partial → урок → q1 ✓ → q2 ✓…
[тренер] Из каких элементов состоит требование в
спецификации OpenSpec? Назови ключевые слова формата.
← q3, здесь клиент вышел
✖ сервер-субпроцесс умер вместе с клиентом

$ python mcp_probe.py --student verify-m3b  # часть 2:
НОВЫЙ процесс
<<< type=question · resumed=True · kind=quiz
[тренер] Из каких элементов состоит требование в
спецификации OpenSpec? Назови ключевые слова формата.
← ТОТ ЖЕ вопрос
=== Вердикт квиза: openspec-basics — счёт 1.00 (порог
0.60), зачтён ===
· openspec-basics: счёт 1.00, серия 8, уроков завершено
8
```

✖ **Kill** — сервер-субпроцесс умирает вместе с клиентом

Restart — новый сервер + новый клиент, тот же ученик

resumed=True — граф вернул тот же вопрос q3, доквиз до зачёта

Чекпоинт — **единственное** состояние (ADR-008). Это дословные строки живых логов 03.11.2026, не сгенерированный сценарий.

Тренер учит SDD — и построен по SDD

proposal → спека + дельты → тикеты #2–#15 → implement (TDD) → независимая проверка → archive ✓

4
Четыре change (openspec/changes/): skeleton, console, mcp (архив), pre-submission-alignment

14
тикетов закрыты полным циклом: работа → проверка владельцем → пуш

4/4
openspec validate — спеки живые, критерии приёмки формализованы

На защите проверяющие видят системный подход **в самом репозитории**: спеки → тикеты → коммиты — SDD не только предмет тренера, но и способ его постройки.

Что сделано — и что дальше

Сделано

✅ Фаза 1 — консольный тренер (T1–T7)

✅ Фаза 2 — MCP-прототип: M1 сервер · M2 7 инструментов · M3 смоук+сценарий

✅ FastAPI REST+SSE + /mcp в одном процессе (PSA-T2) · get_student_history (PSA-T3)

✅ 249 тестов · coverage 78.35% (гейт 40%)

✅ Живые прогоны: полный сеанс + рестарт-резюм

Дальше

🎥 Скринкаст ≤10 мин — сценарий готов (docs/screencast-mcp.md)

📤 Сдача: git.standard-data.ru · main · 13.11.2026

🧭 Кандидаты Фазы 3: resume-aware демо-клиент, GitHub API tool

Разработка: `github.com/Alex-Kul777/sdd-trainer`

SDD Trainer · спасибо!

Курс → проект: карта наследования

НОУТБУК КУРСА · РАЗДЕЛЫ	→ ГДЕ ЖИВЁТ В SDD TRAINER (AS-BUILT)
AIA_01 · Вызов API GigaChat, диалоговый формат	`core/llm_manager.py`
AIA_02 · Pydantic, структурная генерация, инструменты	Pydantic-схемы вердиктов, structured output + `tool_choice` (ADR-005)
AIA_03 · LangGraph, human-in-the-loop	Граф assess → route → teach → quiz; interrupt-паузы = диалог с учеником
AIA_04 · Чекпоинтер LangGraph, память	SqliteSaver + `thread_id student:<name>` (ADR-008)
AIA_05 · Tavily, поиск как инструмент, реранк	`search_docs` — живой поиск в уроках и MCP-tool
AIA_06 ⭐ · Агент со сложным роутингом	**Прямой шаблон графа**: 4 маршрута — knows_it · partial · new_topic · clarification
AIA_07 · F1, токенизация; interrupt + Command	Взвешенный токенный F1 квизов (ADR-009); `Command(resume)` в MCP
AIA_08 (PDF) · Архитектура agent harnesses	MCP stdio как harness-протокол (ADR-010)

План (29.10) → реализация: ReAct-цикл заменён детерминированным роутингом — воспроизводимость вердиктов · GigaSearch не понадобился — хватило Tavily · REST+SSE реализованы, MCP смонтирован в тот же процесс — не замена, а второй шов; паттерн interrupt/Command сохранён.
Честно не перенесено из курса: реранкер, pass@k, субагенты, CoT-критик — осознанный выбор, а не пробел.

Ноутбук AIA_01 → SDD Trainer

AIA_01 · Введение в ИИ агентов

ADR-005 · КРИТЕРИЙ 1

НОУТБУК КУРСА		В SDD TRAINER
OpenAI-клиент: api_key + base_url	→	`llm_manager.py: LLMManager.client (lazy)`
chat.completions.create(model, messages)	→	`llm_manager.py: _complete() — единая точка вызовов`
Роли system / user / assistant	→	`llm_manager.py: chat() — сборка сообщений`
LLM stateless — история на клиенте	→	`llm_manager.py: LLMManager.history (in-memory)`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_02 → SDD Trainer

AIA_02 · Инструменты и структурная генерация

ADR-005

НОУТБУК КУРСА		В SDD TRAINER
Pydantic-модель → JSON-схема (tools)	→	`llm_manager.py: _structure_tool() — схема как tool`
tool_choice: форсированный вызов функции	→	`llm_manager.py: llm_call() — structured output`
model_validate_json + ValidationError	→	`llm_manager.py: llm_call() — валидация против схемы`
Инструмент = описание + реализация	→	`tools.py: search_sdd_docs / analyze_student_repo`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_03 → SDD Trainer

AIA_03 · LangGraph

КРИТЕРИЙ 2

НОУТБУК КУРСА		В SDD TRAINER
StateGraph(TypedDict) + add_node/add_edge	→	`graph.py: build_graph() — 5 узлов assess/route/…`
add_conditional_edges + словарь переходов	→	`graph.py: route_after_route — knows_it/partial/…`
compile() графа	→	`graph.py: build_graph() — компиляция с checkpointer`
HITL-узлы: hitl_need → hitl_feedback	→	`graph.py: clarify_node — interrupt-уточнение`
llm_call(dialog, structure=…)	→	`llm_manager.py: llm_call(messages, structure)`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_04 → SDD Trainer

AIA_04 · Состояние и память ИИ агентов

ADR-008 · КРИТЕРИЙ 4

НОУТБУК КУРСА		В SDD TRAINER
TypedDict-состояние графа	→	`graph.py: TrainerState — 12 каналов`
Reducer: Annotated[List, add]	→	`graph.py: log, clarification_answers — накопление`
Чекпойнтер + thread_id	→	`memory.py: SqliteSaver + thread_id student:<name>`
Долгосрочная память между сессиями	→	`memory.py: TrainingStateStore.record_quiz_result()`
Краткосрочная история диалога	→	`llm_manager.py: LLMManager.history`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_05 → SDD Trainer

AIA_05 · Поиск в LLM-агентах и LangGraph

ADR-006 · КРИТЕРИЙ 3

НОУТБУК КУРСА		В SDD TRAINER
tavily_client.search(query, max_results)	→	`tools.py: search_sdd_docs() — POST api.tavily.com/search`
Нормализация выдачи к единому виду	→	`tools.py: _search_via() — [{title, url, content}]`
Поиск как узел графа: need → search	→	`graph.py: teach_node — DocsDecision → docs_search`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_06 ⭐ → SDD Trainer

AIA_06 ⭐ · Практика разработки ИИ агентов КРИТЕРИЙ 5

НОУТБУК КУРСА		В SDD TRAINER
Роутинг-агент: routing_node + переходы	→	graph.py: route_node + add_conditional_edges
HITL-уточнение запроса	→	graph.py: clarify_node — interrupt
Bounded loop — цикл уточнений ограничен	→	graph.py: MAX_CLARIFICATION_ROUNDS
Инструменты в узлах агента	→	graph.py: assess_node (repo_analyzer)

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_07 → SDD Trainer

AIA_07 · Сбор метрик и деплой ИИ агентов

ADR-009 · КРИТЕРИЙ 6

НОУТБУК КУРСА		В SDD TRAINER
Токенизация + Precision/Recall/F1	→	`quiz.py: tokenize() + answer_f1() — SQuAD-style F1`
Деплой через FastAPI: POST /invoke	→	`app.py: /sessions/{student}/start, /answer (pydantic)`
graph.stream(stream_mode=updates)	→	`app.py: _sse_events() — SSE-стрим шагов`
interrupt + Command(resume=…)	→	`app.py: answer() — resume с чекпоинтом`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Ноутбук AIA_08 → SDD Trainer

AIA_08 · Архитектура современных harnesses (PDF)

ADR-010/001 · КРИТЕРИЙ 6

НОУТБУК КУРСА		В SDD TRAINER
MCP-сервер над агентом	→	`mcp/server.py: create_server() — FastMCP над графом`
@mcp.tool — типизированные инструменты	→	`mcp/server.py: start_session / answer_* (7 шт.)`
Локальный MCP через stdio	→	`mcp/__main__.py: python -m sdd_trainer.mcp`
Streamable HTTP рядом с REST	→	`app.py: mount /mcp — один процесс, второй шов`

каждая строка проверена по коду репозитория (факт-чек: блок → файл/функция)

Три прямых наследования

Роутинг → скелет тренажёра

AIA_06: knowledge_check → routing → clarification / knowledge

Проект: assess → route → clarification / teach

Узлы учебного агента стали ядром графа — критерий 5

Чекпоинтер → фича рестарта

AIA_04: упражнение с чекпоинтером LangGraph

Проект: SqliteSaver + thread_id — kill посреди квиза → **resumed=True**

Упражнение стало фичей — слайд 9, ADR-008

interrupt + Command → MCP

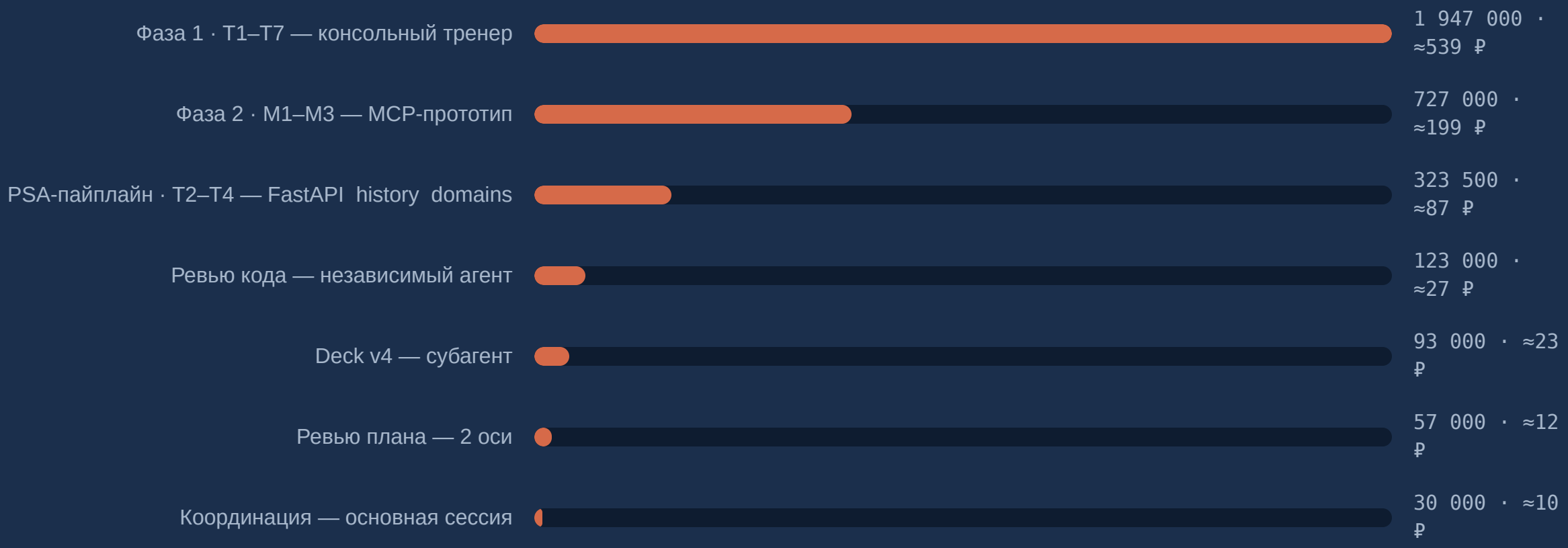
AIA_07: стриминг с HITL через interrupt и Command

Проект: `server.py:296 — Command(resume=answer)`

Каждое сообщение ученика продолжает прерванный граф; F1 → вердикт квиза

Курс не остался теорией: **каждый паттерн из ноутбуков — строка в коде тренажёра**

Бюджет разработки агентом

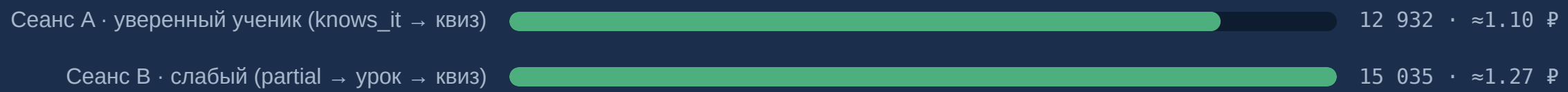


≈3.3M токенов · ≈900 ₽

весь проект разработан ИИ-агентом — 17 сессий, in 2.55M + out 0.75M

Источник: кумулятивные счётчики OpenClaw session_status по 17 сессиям, 04.11.2026 · Тариф cloud.ru (GLM-5.3): 173.15 ₽/1M входных · 606.05 ₽/1M генерируемых · Кэш: 86–97% hit (~55M попаданий) — тарифицируется отдельно, дешевле · API предоставлен в рамках курса — фактических списаний нет.

Стоимость обучения: один сеанс ≈ 1 ₽



≈1.2 ₽ за сеанс — ~14k токенов GigaChat-3-Pro, 4–5 вызовов

Весь бюджет разработки (≈900 ₽) — это **≈750 сеансов обучения**

Живой замер 05.11.2026: два полных сеанса через граф (калибровка → маршрут → квиз → вердикт), живой Tavily в уроке · Тариф cloud.ru GigaChat-3-Pro: 73.03 ₽/1M входных · 176.39 ₽/1M генерируемых · API в рамках курса — фактических списаний нет.